Tonet Lorenzo

https://github.com/LorenzoTonet/Graph-pond-Network

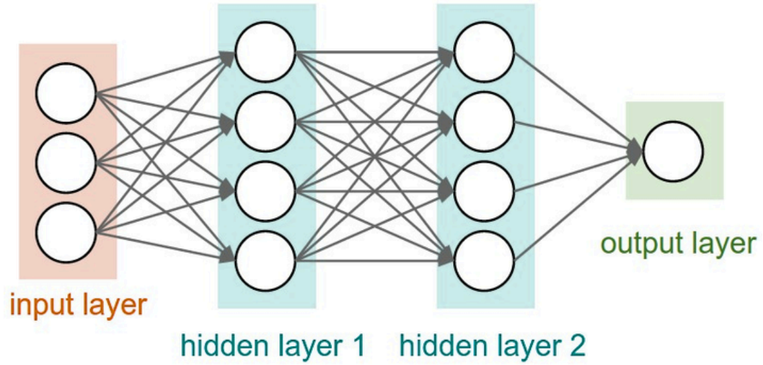
Pondering Graph Neural Networks

Deep Learning - Final Project

Framework Introduction

Main problem

As we intend Multi-Layer Perceptron (MLP) today, we have an input layer, a **fixed number** of hidden layers



Each layer correspond of a "step of computation" of an input and in a certain way, it embodies the complexity of the problem being learned.

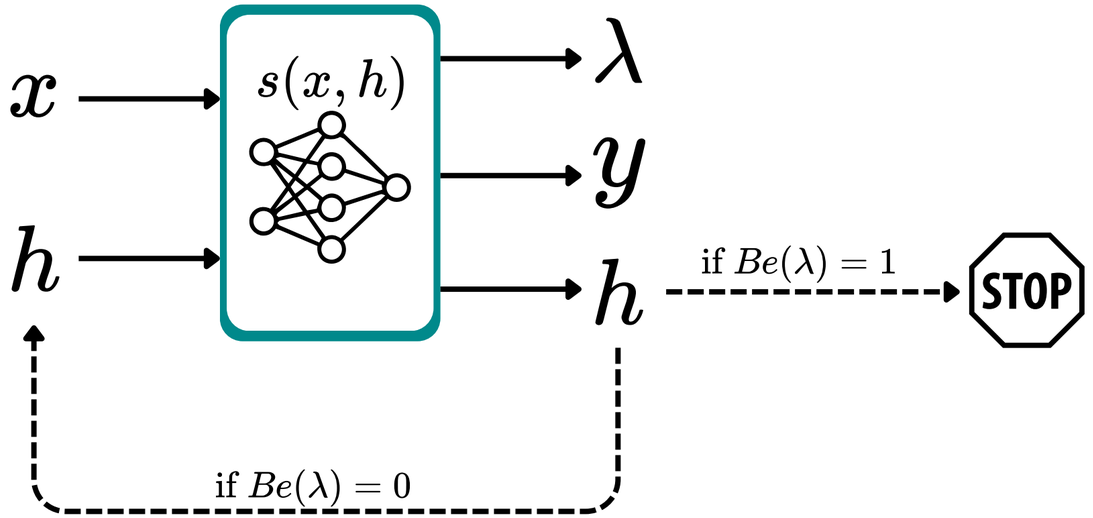
Research question:
What if we were able to learn how difficult is a problem and in consequece do "more computation" for difficult ones and less for the easier?

Choosen architecture

In 2021 Google DeepMind introduced an algorithm for **adaptive computation** called PonderNet. It works like a recursive step function which update its prediction/embedding of the input every time

$$s(x, h_t) = y_{t+1}, h_{t+1}, \lambda_{t+1}$$

prediction *embedding* *halting probability*



8th ICML Workshop on Automated Machine Learning (2021)

PonderNet: Learning to Ponder

Andrea Banino*
DeepMind
London, UK
abanino@deepmind.com

Jan Balaguer*
DeepMind
London, UK
jan@deepmind.com

Charles Blundell
DeepMind
London, UK
cblundell@deepmind.com

Abstract

In standard neural networks the amount of computation used grows with the size of the inputs, but not with the complexity of the problem being learnt. To overcome this limitation we introduce PonderNet, a new algorithm that learns to adapt the amount of computation based on the complexity of the problem at hand. PonderNet learns end-to-end the number of computational steps to achieve an effective compromise between training prediction accuracy, computational cost and generalization. On a complex synthetic problem, PonderNet dramatically improves performance over previous adaptive computation methods and additionally succeeds at extrapolation tests where traditional neural networks fail. Also, our method matched the current state of the art results on a real world question and answering dataset, but using less compute. Finally, PonderNet reached state of the art results on a complex task designed to test the reasoning capabilities of neural networks.

1. Introduction

The time required to solve a problem is a function of more than just the size of the inputs. Commonly problems also have an inherent complexity that is independent of the input size: it is faster to add two numbers than to divide them. Most machine learning algorithms do not adjust their computational budget based on the complexity of the task they are learning to solve, or arguably, such adaptation is done manually by the machine learning practitioner. This adaptation is known as pondering. In prior work, Adaptive Computation Time (ACT; Graves, 2016) automatically learns to scale the required computation time via a scalar halting probability. This halting probability modulates the number of computational steps, called the "ponder time", needed for each input. Unfortunately ACT is notably unstable and

*. contributed equally

Training

In the training phase the computation is done by letting the network do a certain number of steps (what will become the maximum number of steps) and the loss will penalize both the actual prediction of the model and the probability of reaching it. The loss will be so defined as

Hyperparameters

The task loss itself (ie. cross-entropy for classification and so on)

$$\mathcal{L}(\lambda_p, \beta) = \sum_{i=1}^{n_{\max}} p_n(i)\, \mathcal{L}(y, y_{\text{true}}) \; + D_{KL}\left[p_n || q(\lambda_p)\right]$$

This represent "*the probability of having halted at step i*" and it is the reason why in training all steps must be done

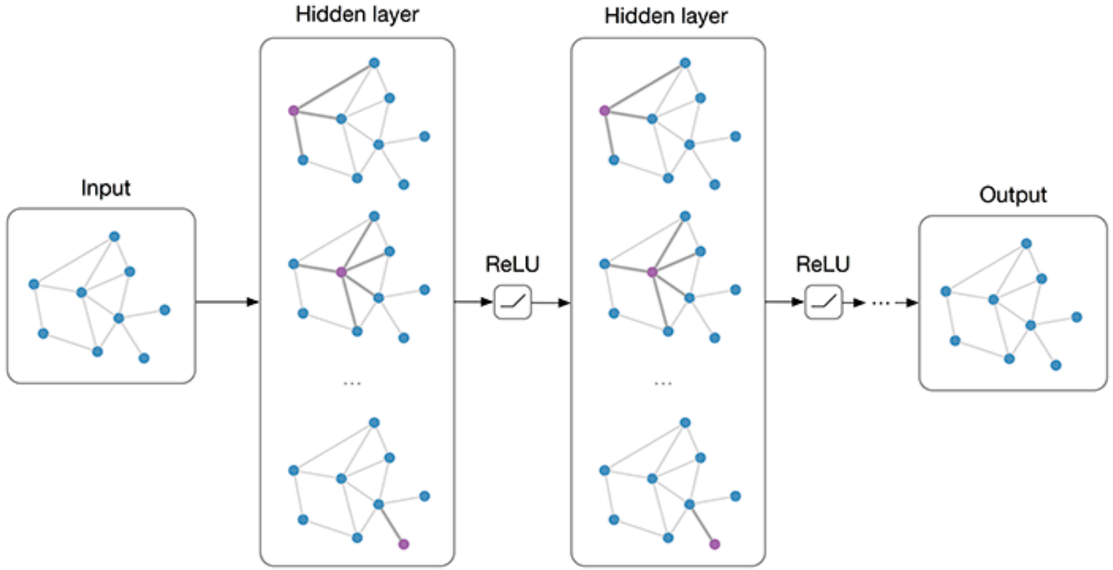
$$p_n(i) = \lambda_i \prod_{k=1}^{i} (1 - \lambda_k)$$

This term act as a regularization of the distribution toward a geometric prior of parameter λ_p

Deep Learning on graphs

Graph Neural networks

Graph neural networks (also called Graph Convolutional networks) are a particular architecture built to perform tasks where the input is structured as a graph represented by an adjency matrix and a set of node and edge features. How the name suggest, GCNs perform a fundamental operation: the **message passing**, a particular adaptation of convolution for graphs



Exactly as CNNs, GCNs perform in an alternated way two key operations, **aggregation** and **pooling**

The message passing

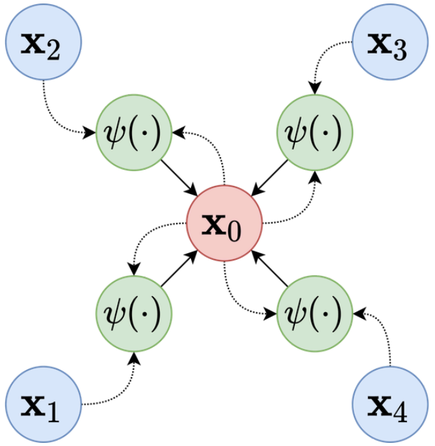
As we seen before, the operation that represent the convolution in this frame is the message passing so every layer perform one message passing. the output of a graph convolutional layer for a node u is formally defined as:

$$h_u = \phi_\theta \left(x_u, \bigoplus_{v \in \mathcal{N}_u} \psi_\theta(x_u, x_v, e_{uv}) \right)$$

This is called **update function**, the one which updates the node representation based on the messages from the neighbours

permutation invariant **aggregation operator**

This is the **message function**, it encodes the signal from v to u

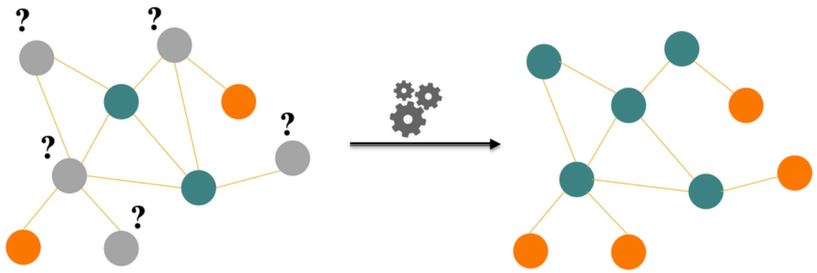


Each message-passing layer increases a node's receptive field by one hop.

Tasks on graphs

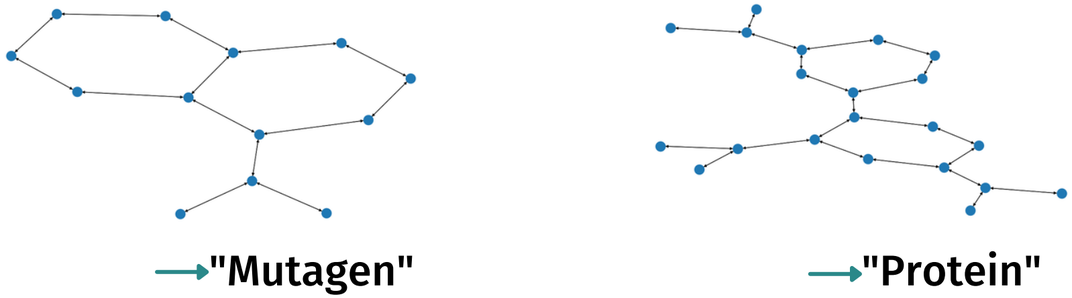
Due to the flexibility of the graph structure, many task can be performed over data structured as this. In this work the focus will be put on 2 particular kind of tasks:

Node classification



Node classification is the prediction of a **label or class for each node** in a graph. It uses both the individual features of a node and the structural connections (edges) to its neighbors to fill in missing labels

Graph classification

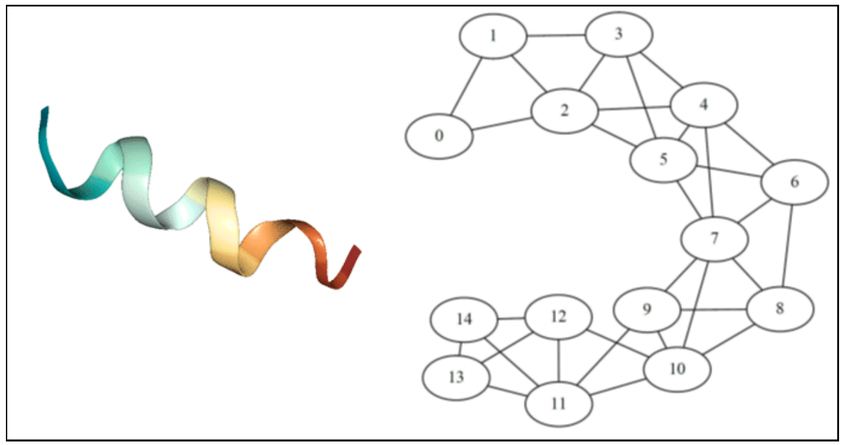


Graph classification is a task that assigns a **single categorical label or continuous property to an entire graph** by analyzing its global topology and local structural patterns.

Core Idea

Flexible message passing

Message passing is inherently local, propagating information only to one-hop neighbors, while **the target information may depend on interactions between distant nodes**. On the other hand for other kinds of graph/nodes the key information can depend only on nearby nodes.



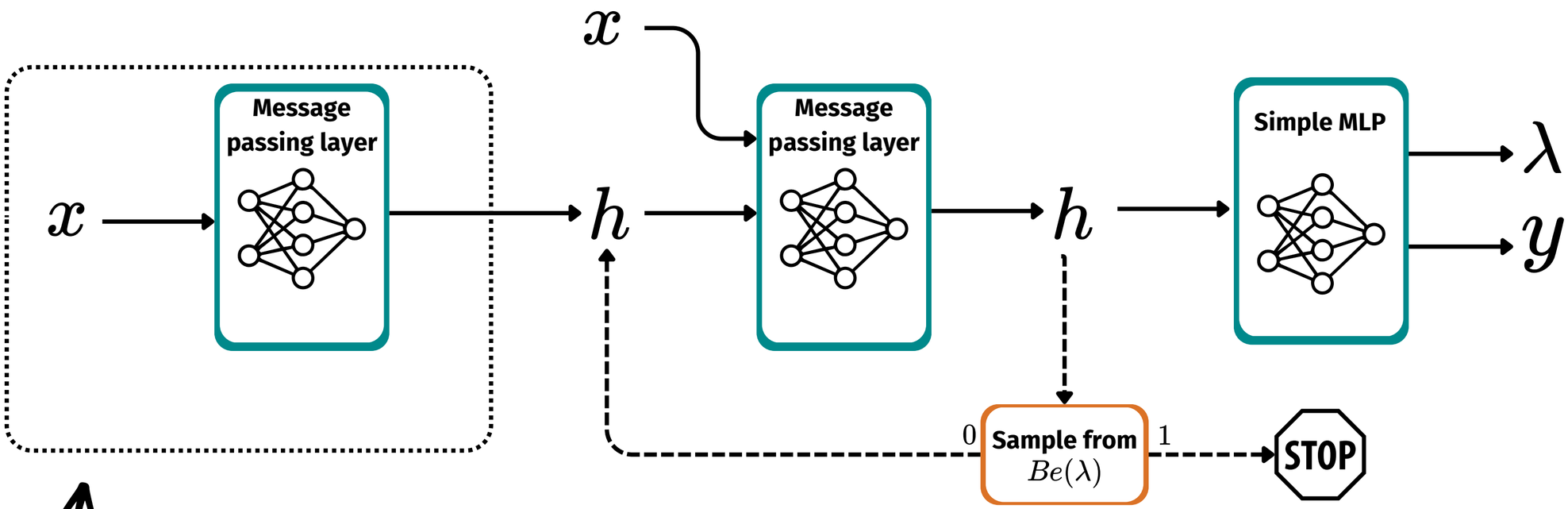
Maybe the fact that this structure is a protein is encoded in the relation between nodes 0 and 12. Maybe the key part is the neighbourhood of node 9

The main idea of this work is to combine the PonderNet framework with GNNs, where each computation step represents one additional graph distance over which information is propagated. Consequently, the model can **adapt the propagation depth** required to gather the information necessary for the final label prediction.

This mechanism can be adapted for the 2 different task:

- for node classification the every node has its own halt step
- for graph classification the MP distance is relative to the entire graph

Final architecture

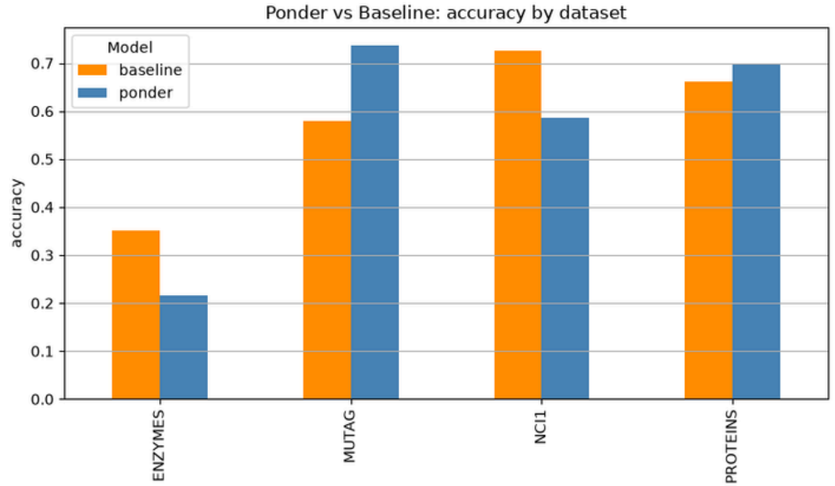


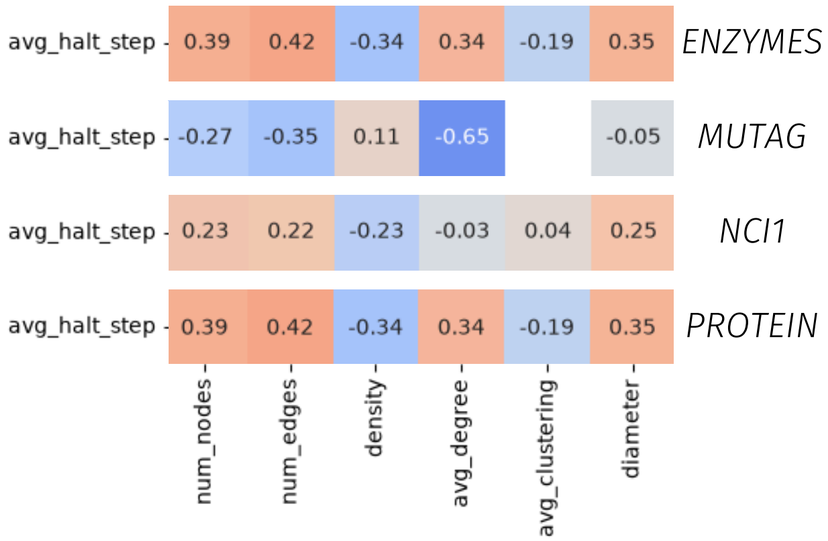
This step is done only the first time to bring the features of the nodes in a desired latent space.

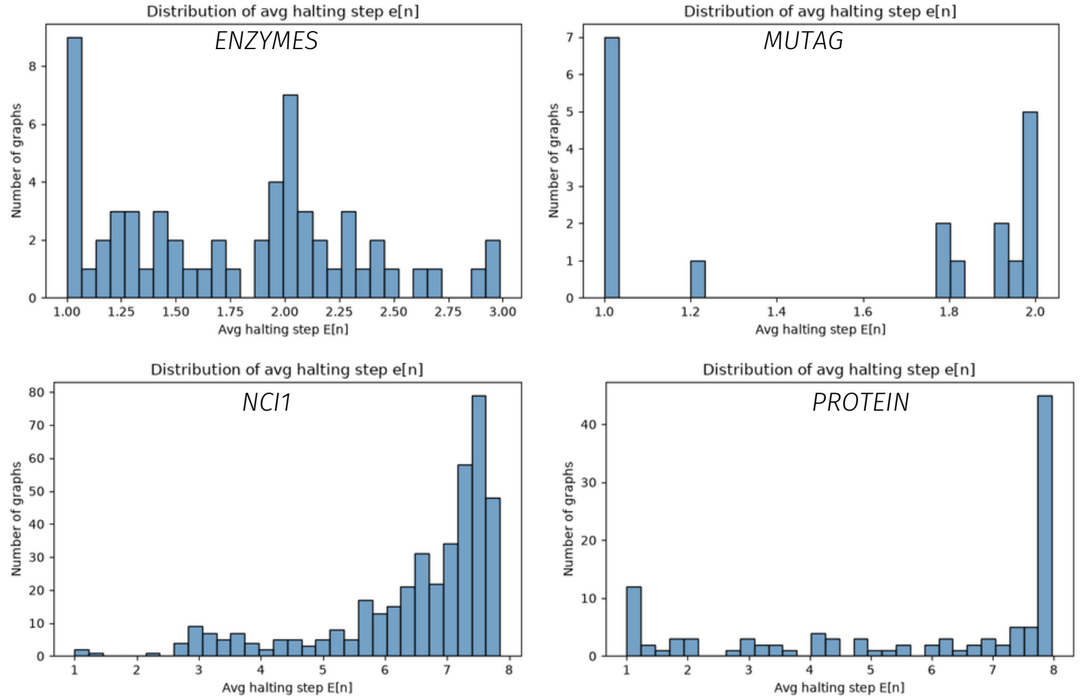
In the node classification task, the HALT is equivalent to **stop the update of the embedding** for a single node. The message passing contiunes but the node will no more update

Results

Graph classification results



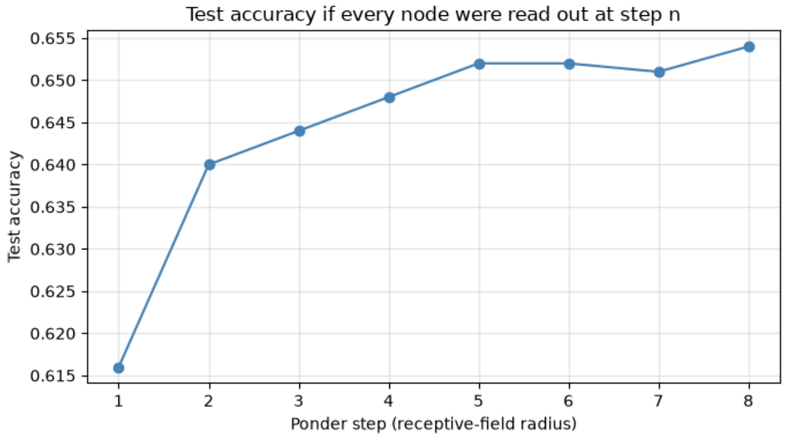


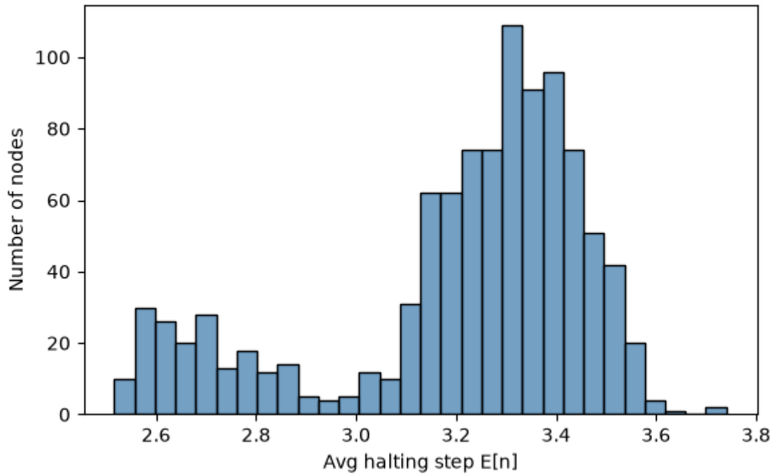


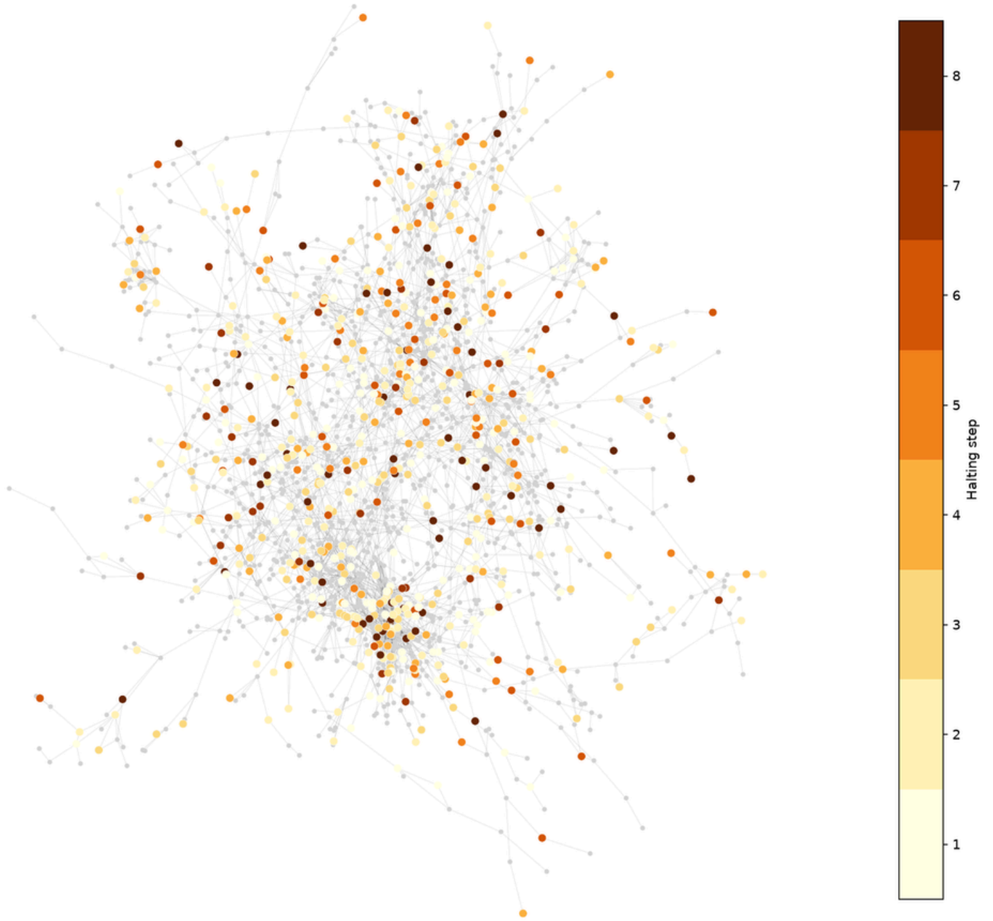
Dataset	Avg. # Nodes	Avg. # Edges	Avg. Degree	# Samples
ENZYMES	32.63	62.14	3.86	600
MUTAG	17.93	19.79	2.19	188
NCI1	29.87	32.30	2.16	4110
PROTEINS	39.06	72.82	3.73	1113

Node classification results

Dataset: CiteSeer



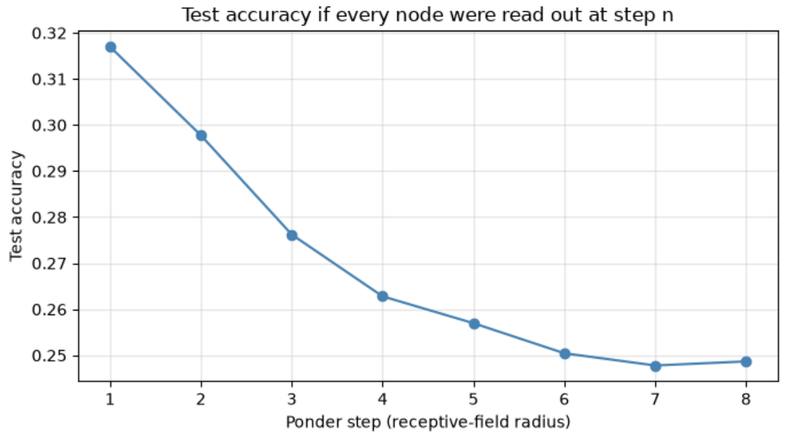


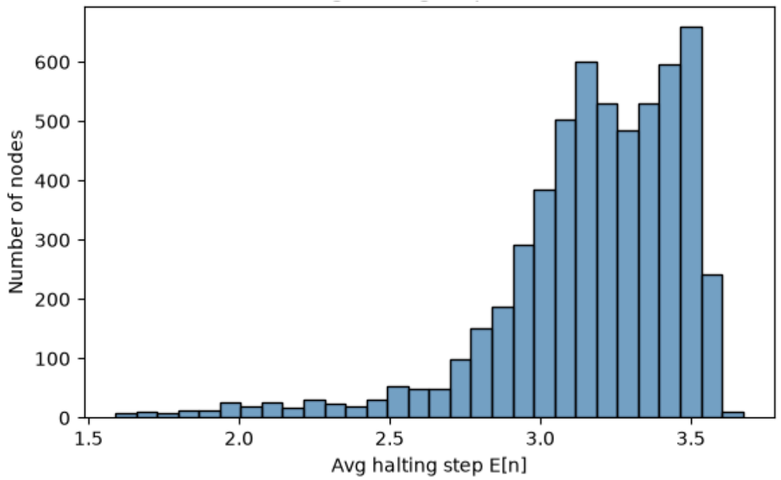


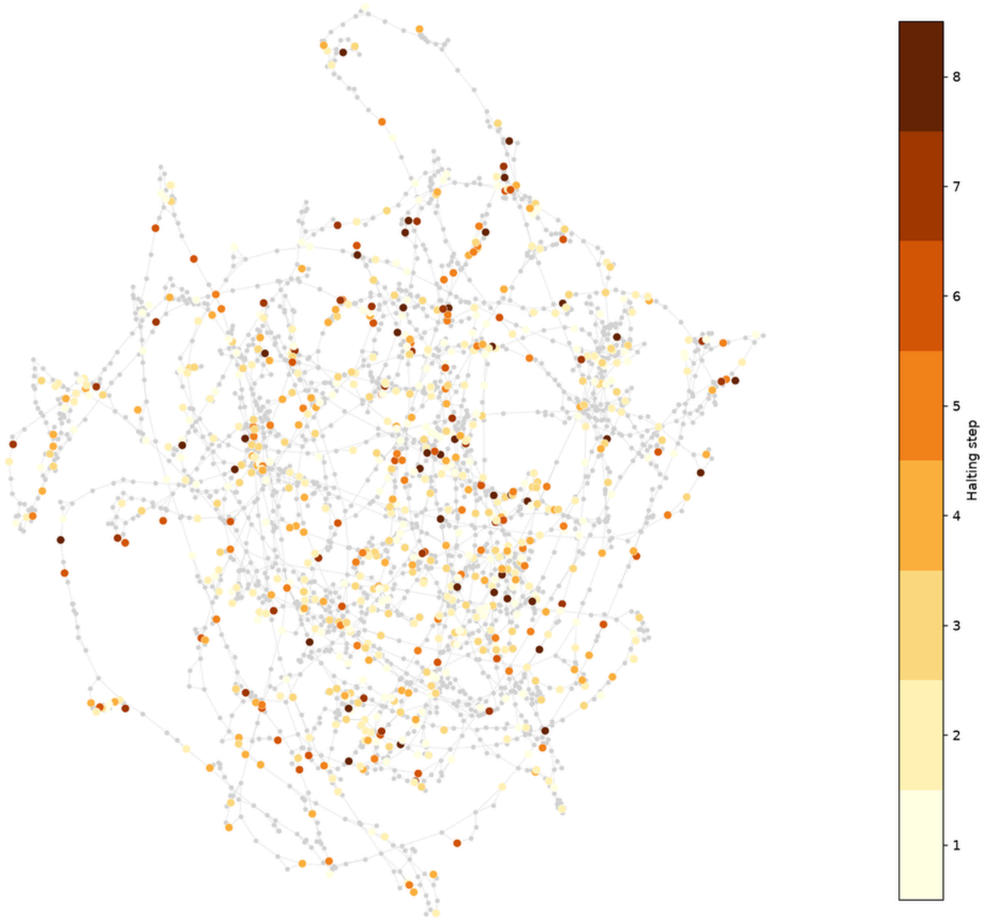
The bias term pushes the number of steps to 5

Node classification results

Dataset: RomanEmpire

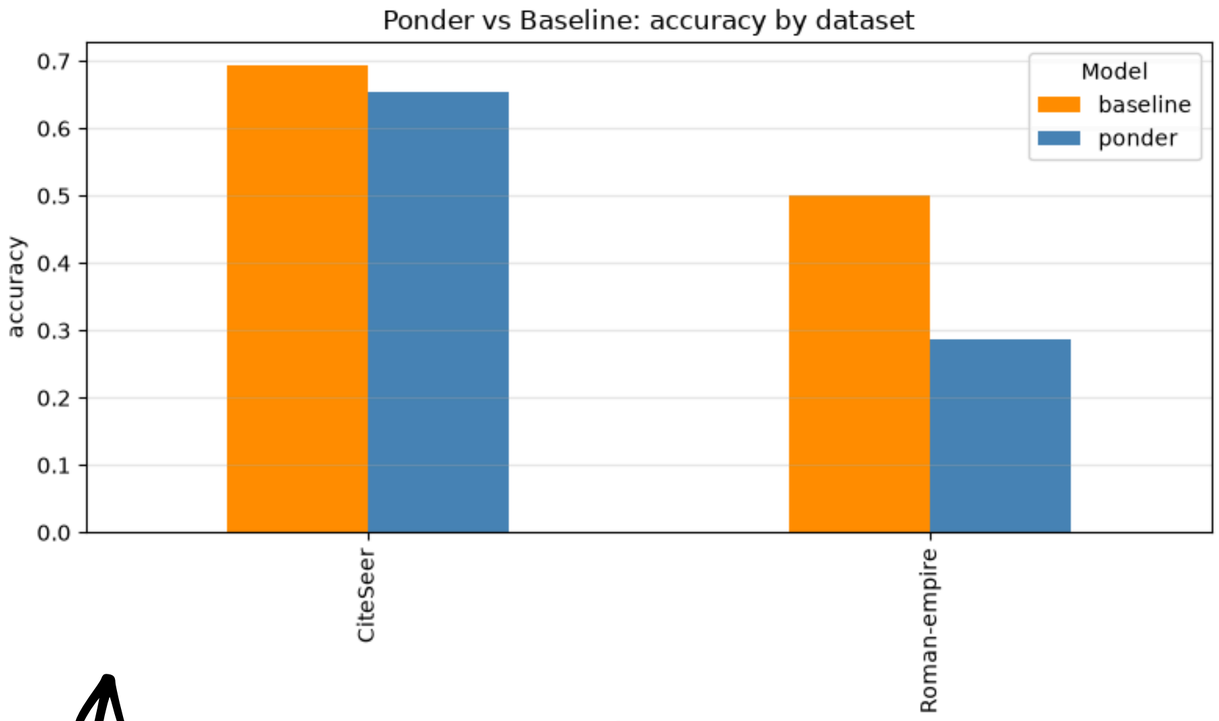




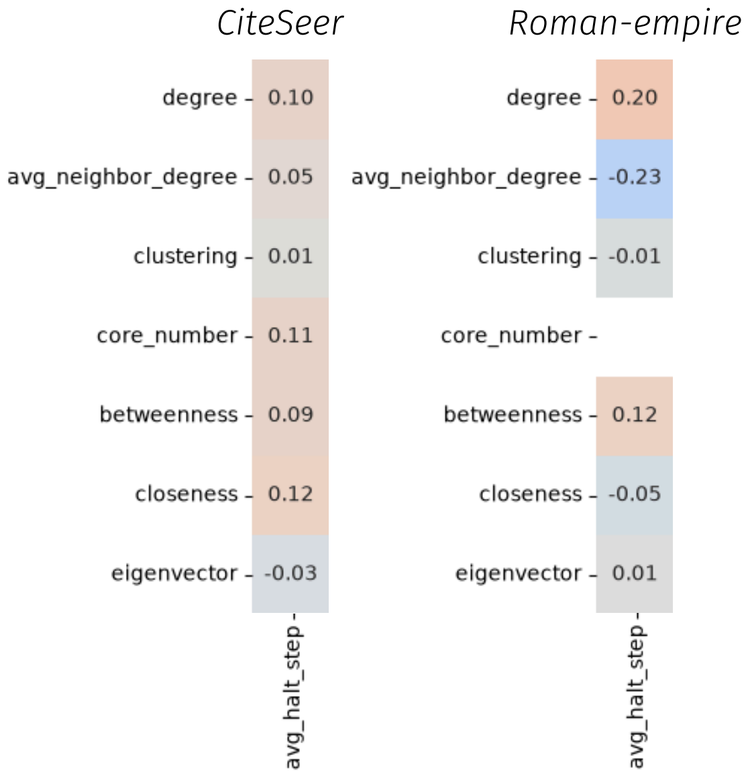


The bias term pushes the number of steps to 5

Node classification results



Despite the higher number of steps, for a NN is more difficult to solve different problems once.



No significant correlation for the node features

The End

Tonet Lorenzo

https://github.com/LorenzoTonet/Graph-pond-Network